Git 版本控制

For Antigravity

參考資料
：https://vscode.com.tw/docs/sourcecontrol/overview#google_vignette

Agent

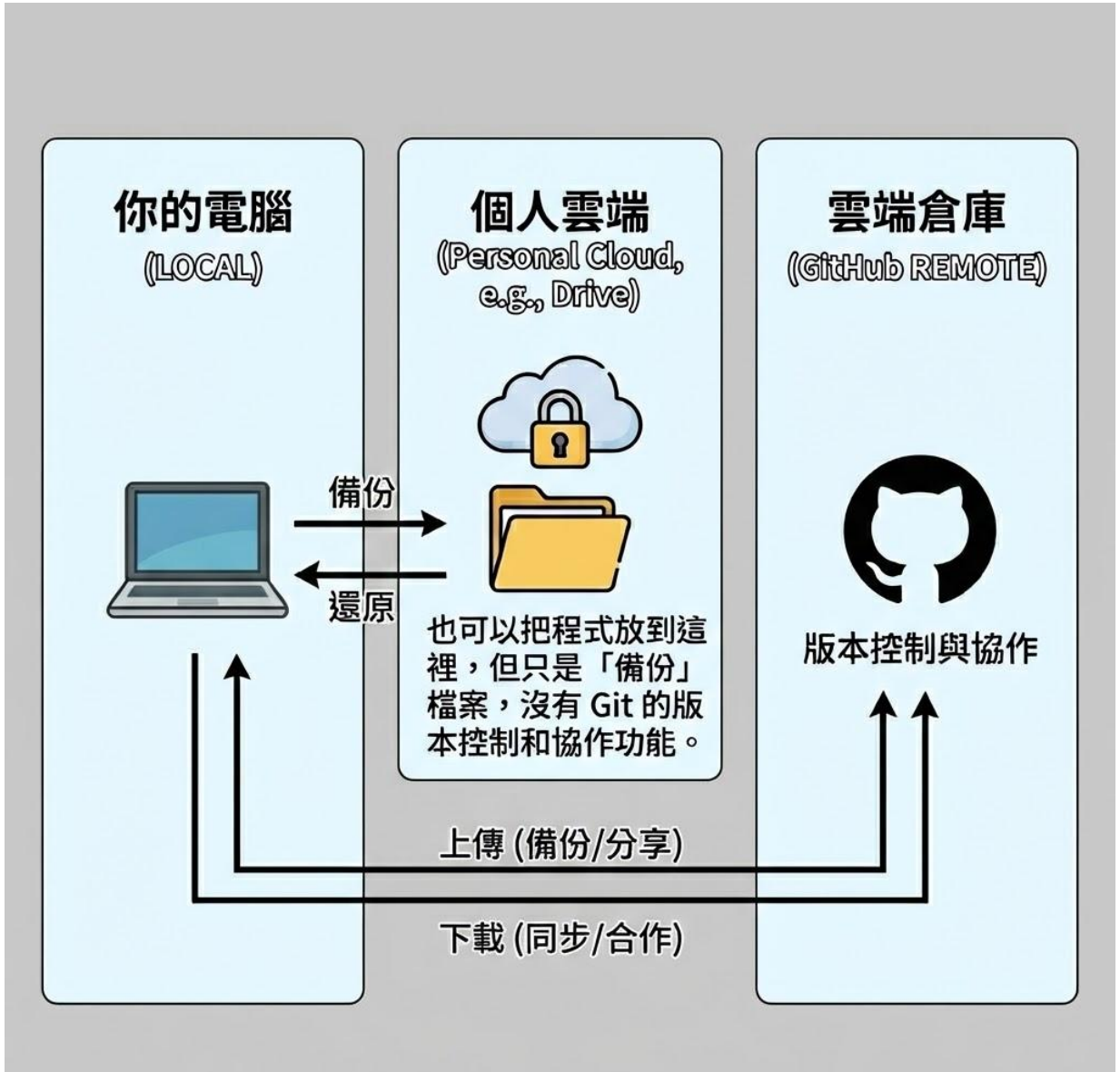
01. Git 是什麼？

02. Antigravity 內建操作

03. Git 實作練習

04. 作業繳交

Git 是團隊合作最常用的溝通工具之一

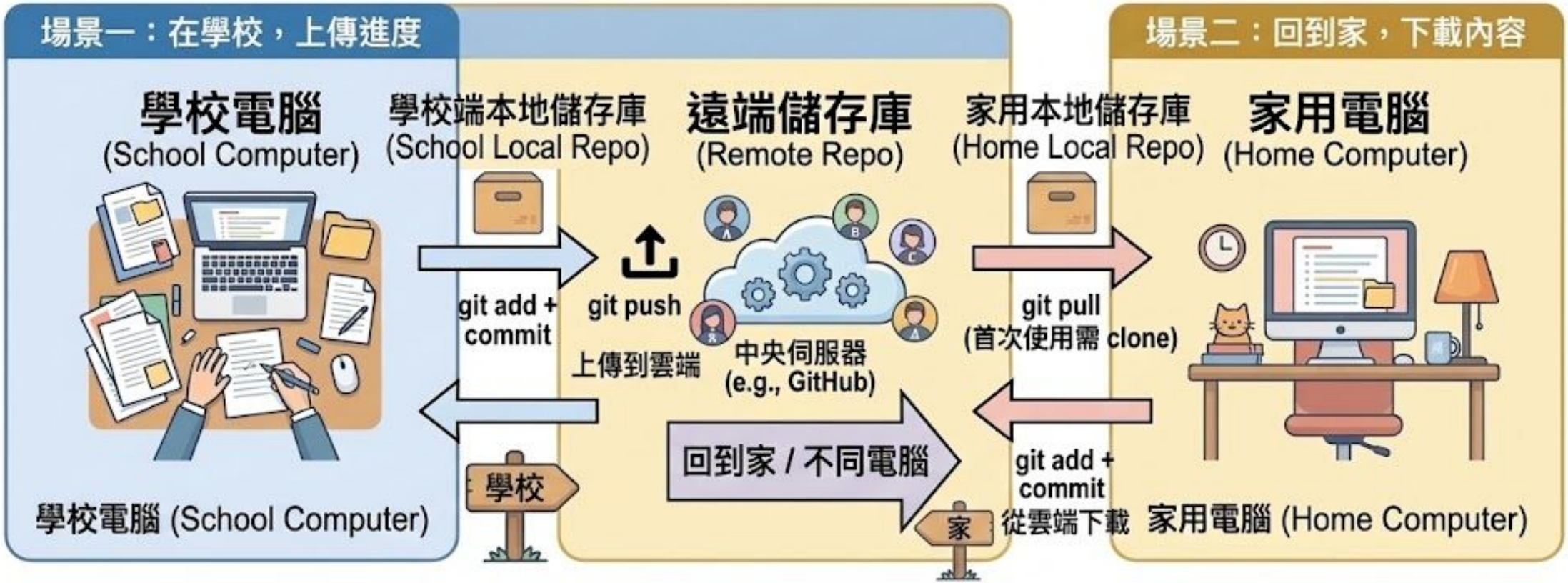


Git 有什麼優點，為什麼大家喜歡用它 ？

Git 常見使用情境

- 本地工作區 (Local Work Space)：就是你電腦上開著編輯器的那個資料夾，你正在這裡改程式碼。
- 本地儲存庫 (Local Repo)：是你電腦上的一個秘密檔案櫃，用來存放你「確認好、貼上標籤」的各個版本。它也在你的電腦上，即使沒網路也能用。
- 遠端儲存庫 (Remote Repo)：就是像 GitHub 這樣的網路伺服器，用來備份你的「本地儲存庫」並與他人共享。
- 關係：你把工作區做好的東西「存入」本地儲存庫 (commit)，確認沒問題後，再「上傳」到遠端儲存庫 (push) 分享給別人。想拿別人的東西，就從遠端「下載」(pull) 回來。

Git 延伸：從學校電腦 Push 後，如何在回到家下載



- 工作區 (School/Home Work Space)：就像是你在電腦上打開編輯器所在的資料夾。
- 學校本地儲存庫 (School Local Repo)：是你在學校電腦上的秘密版本檔案櫃，存放你在學校編輯的所有版本紀錄。
- 家用本地儲存庫 (Home Local Repo)：是你在家裡電腦上的另一個秘密案櫃，存放你家的所有版本紀錄。連線後可同步雲端版本。
- 遠端儲存庫 (Remote Repo)：它是像 GitHub 這樣的網路中央倉庫，連接並同步你在不同電腦上的檔案紀錄。
- 關係圖解：從學校電腦 (School Computer) 推送 (Push) 到雲端，回家後從雲端拉取 (Pull/Clone) 到家用電腦 (Home Computer)。
- 回家打打開電腦，你只需要進行「拉取 (pull/clone)」操作，檔案和歷史紀錄就會同步到你的家用電腦中。

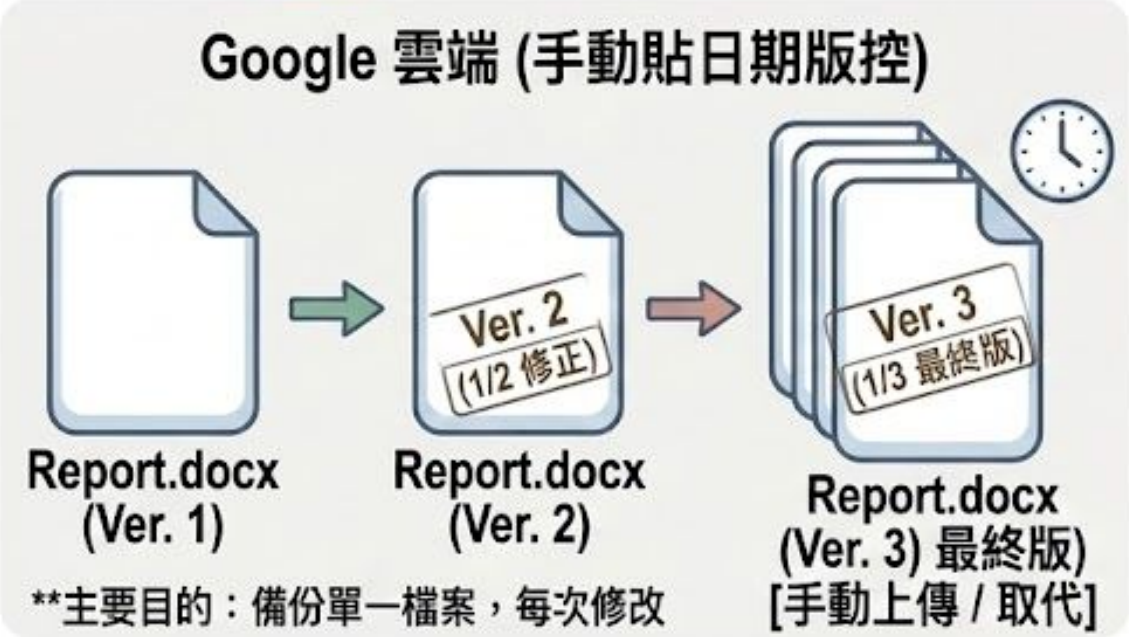
Git (版本控制) 與 Google 雲端 (檔案備份) 有什麼不同？

比較項目	Git (版本控制)	Google 雲端 (檔案備份)
• 管理重心	專案的歷史變更紀錄	最新的單個檔案
• 追溯功能	可隨時回到任意時刻的版本。(看左圖分支)	通常只能找回有限次數的舊版。(看右圖取代)
• 多人協作	適合多人同時編輯程式碼，自動解決衝突。	多人同時編輯同一份檔案易發生覆蓋。
• 關係圖解：	Git 是一個會「記得所有變更」的精準記錄器。你甚至可以用它來寫程式碼分支實驗。	Google 雲端是一個「幫你搬運檔案」的雲端倉庫。最新檔案永遠在最上面。

檔案管理大對決：Google 雲端 (日期版控) vs. Git (真正的版本控制)

Google Cloud

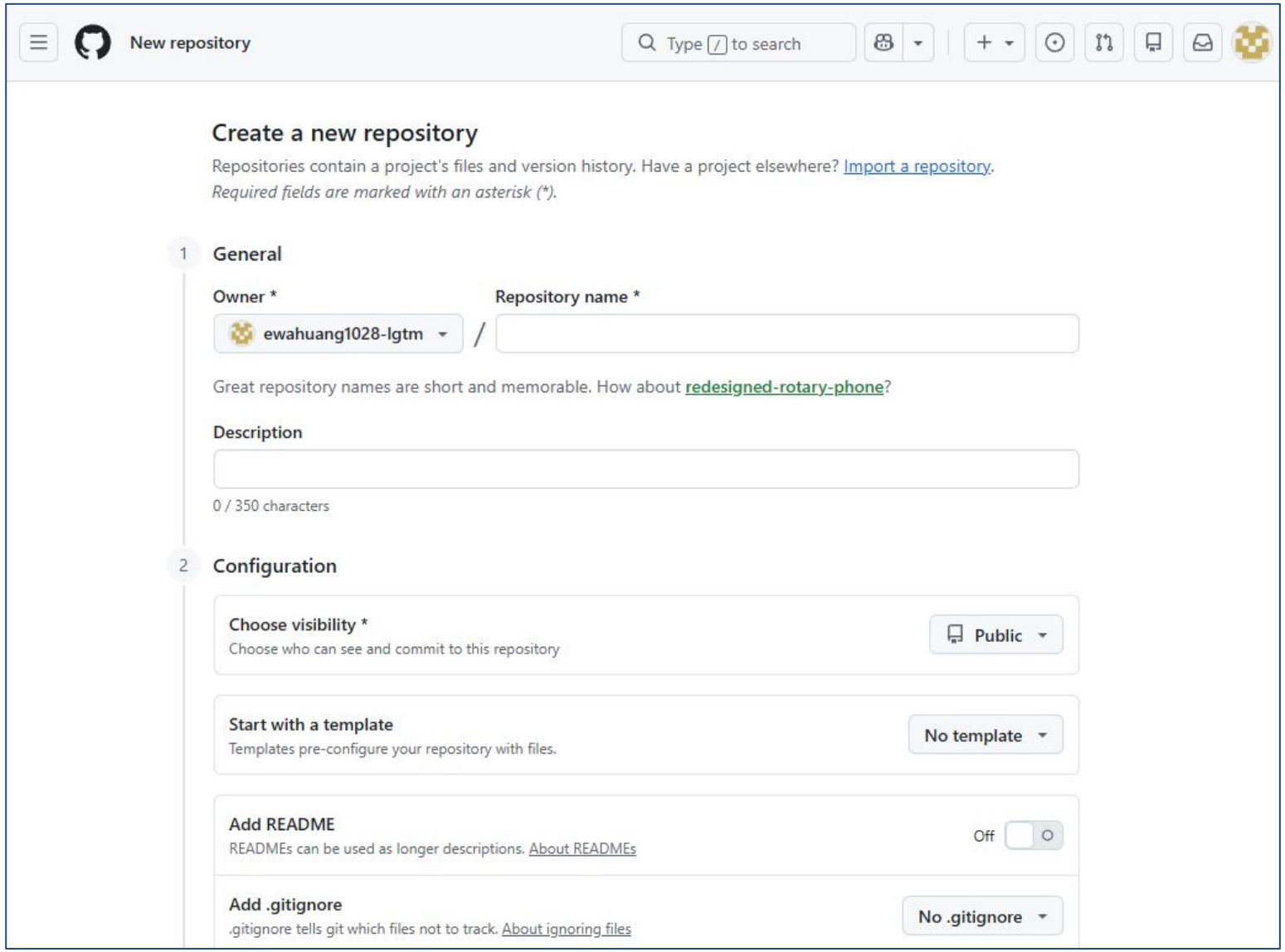
git



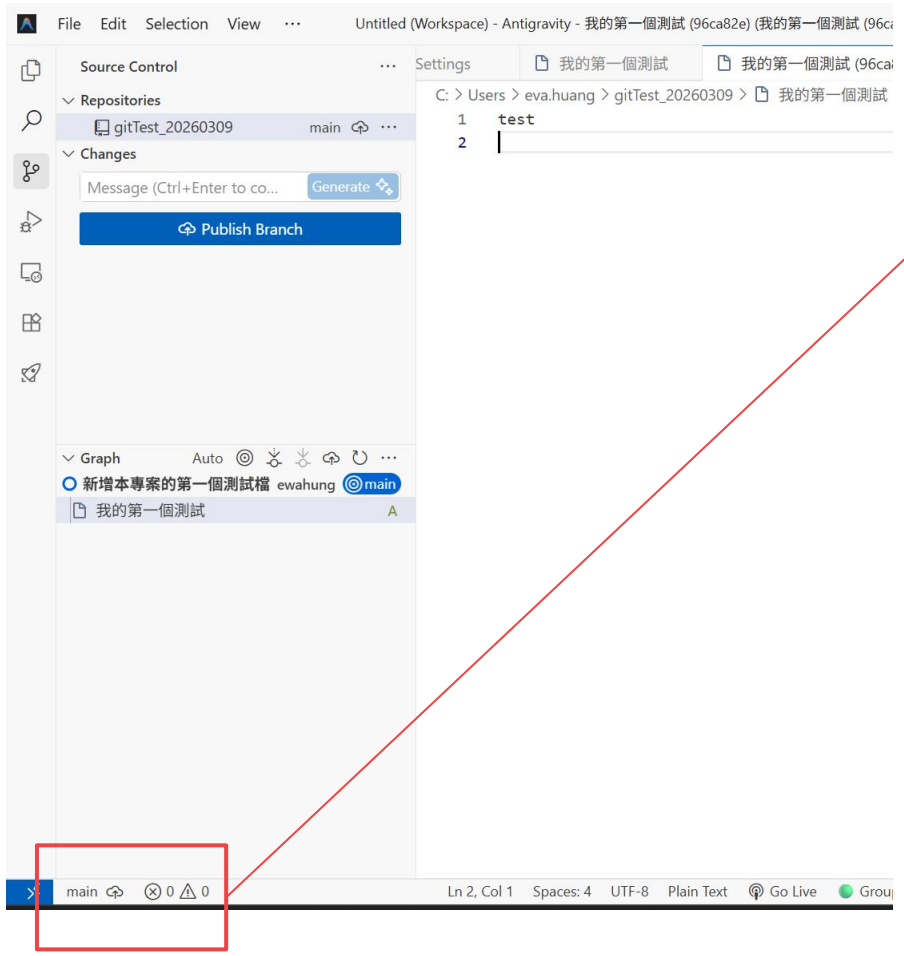
比較項目	Google 雲端 (手動貼日期)	Git (真正的版本控制)
• 管理重心	單一檔案的「最後狀態」	專案的完整「歷史變更紀錄」
• 版本功能	手動命名分類，資訊零散。	自動記錄誰、什麼時間、改了哪幾行。
• 回溯內容	只能找到特定的「備份檔案」。	可隨時回到任意時刻的所有檔案。
• 多人協作	適合少人，手動日期易發生覆蓋。	適合多人同時編輯程式碼，自動解決衝突。
• 關係圖解：	Google 雲端就像一個單純搬運檔案的倉庫 (看左圖取代)	Git 是一個會「記得所有變更」的精準記錄器 (看右圖分支)

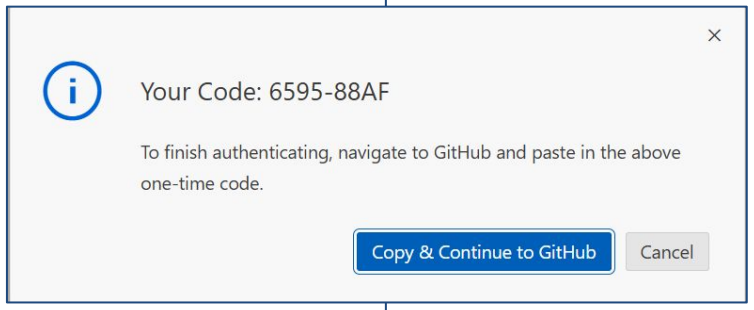
1. GitHub 註冊

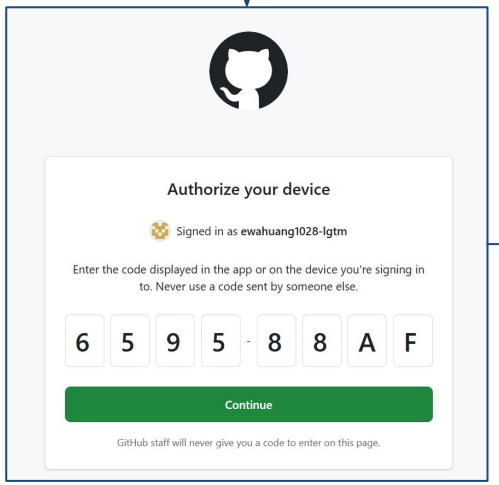
http://github.com

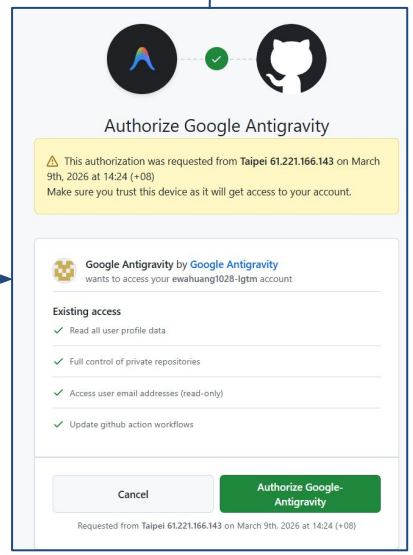


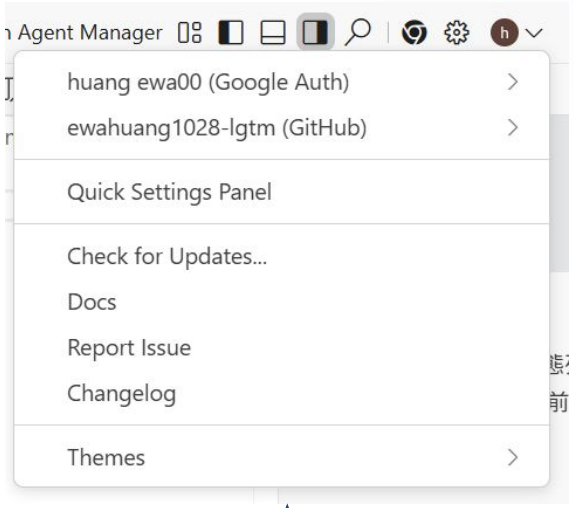
如何在Antigravity登入











Source Control

Repositories

gitTest_20260309 main

Changes

Message (Ctrl+Enter to co... Generate

Publish Branch

Graph Auto

新增本專案的第一個測試檔 ewahung main

我的第一個測試 A

Settings | 我的第一個測試 | 我的第一個測試 (96ca82e)

C: > Users > eva.huang > gitTest_20260309 > 我的第一個測試

```
1  test
2
```

gitTest_20260309

Publish to GitHub private repository ewahuang1028-lgtm/gitTest_20260309

Publish to GitHub public repository ewahuang1028-lgtm/gitTest_20260309

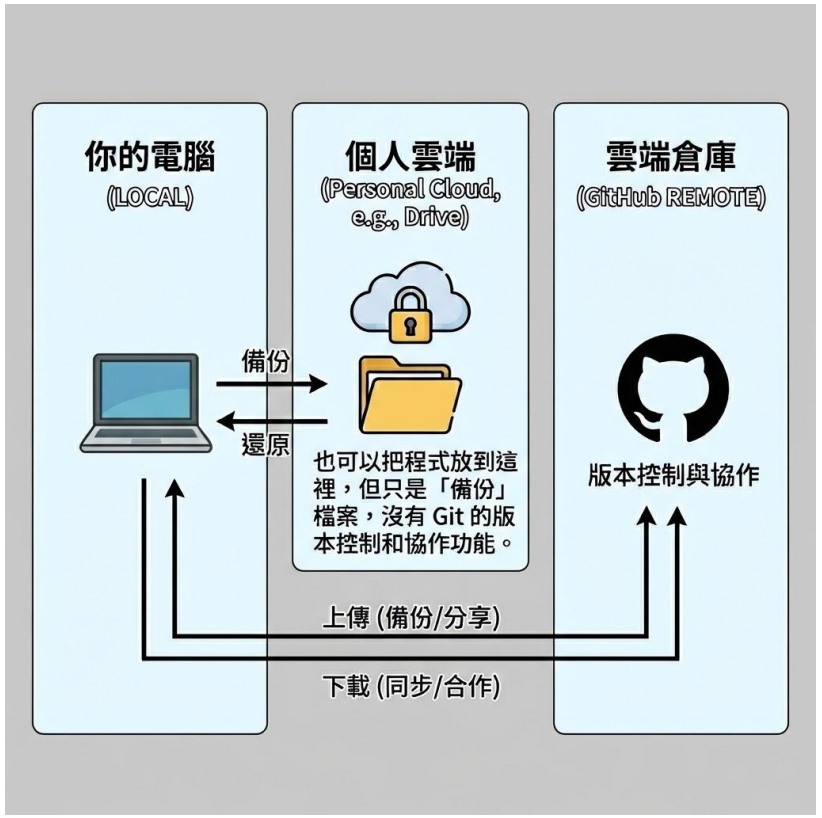
想自用還是公開？

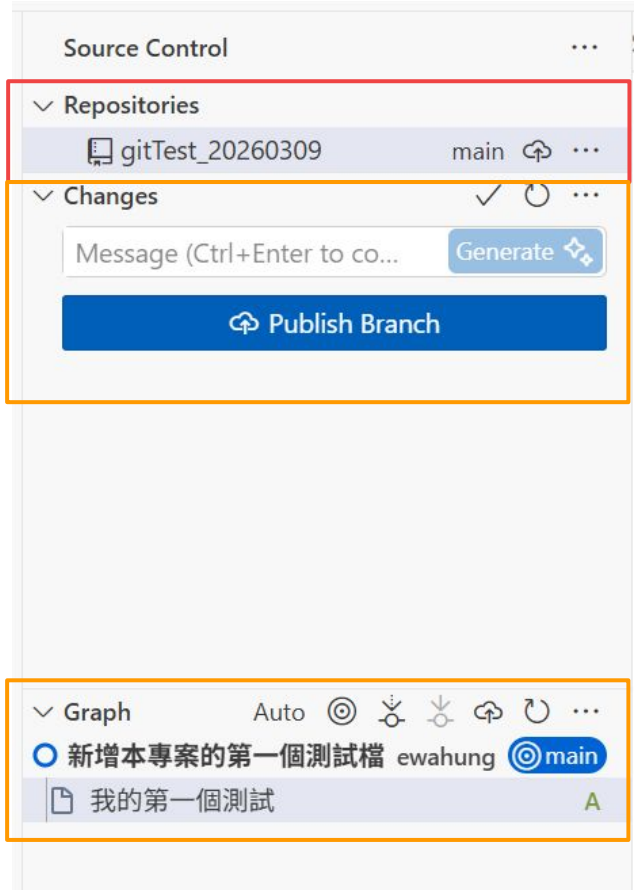
存檔點 (Save Point) 機制

把 Git 想像成遊戲中的存檔機制：

- **避免「全盤皆輸」：** 即使新功能寫爛了，隨時可以讀取上一個存檔。
- **歷史追蹤：** 清楚知道誰在什麼時候，為了什麼目的修改了程式碼。
- **團隊平行時空：** 多人可以同時存檔而互不干擾。

Repository (倉庫)





專案的「家」（Repository）

存放整個專案與所有歷史紀錄的地方。它包含所有的程式碼檔案、圖片素材、配置文件，以及 Git 用來追蹤變更的所有元數據 (.git 資料夾)。

類型： 本地倉庫 (Local) 與 遠端倉庫 (Remote)。

Commit (提交)

「存檔點」

當完成一個功能或修好一個 Bug，就做一次 Commit。

未來隨時可以「讀檔」回到這個狀態。每一次 Commit 都有一個唯一的 ID 與說明訊息。

Remote (遠端)

雲端備份與協作中心

通常指 **GitHub** 或 **GitLab**。

這是用來備份存檔與同學協作的雲端倉庫。讓存檔不只存在於單機，而是全世界都能讀取（如果您允許的話）。

Local Repositories

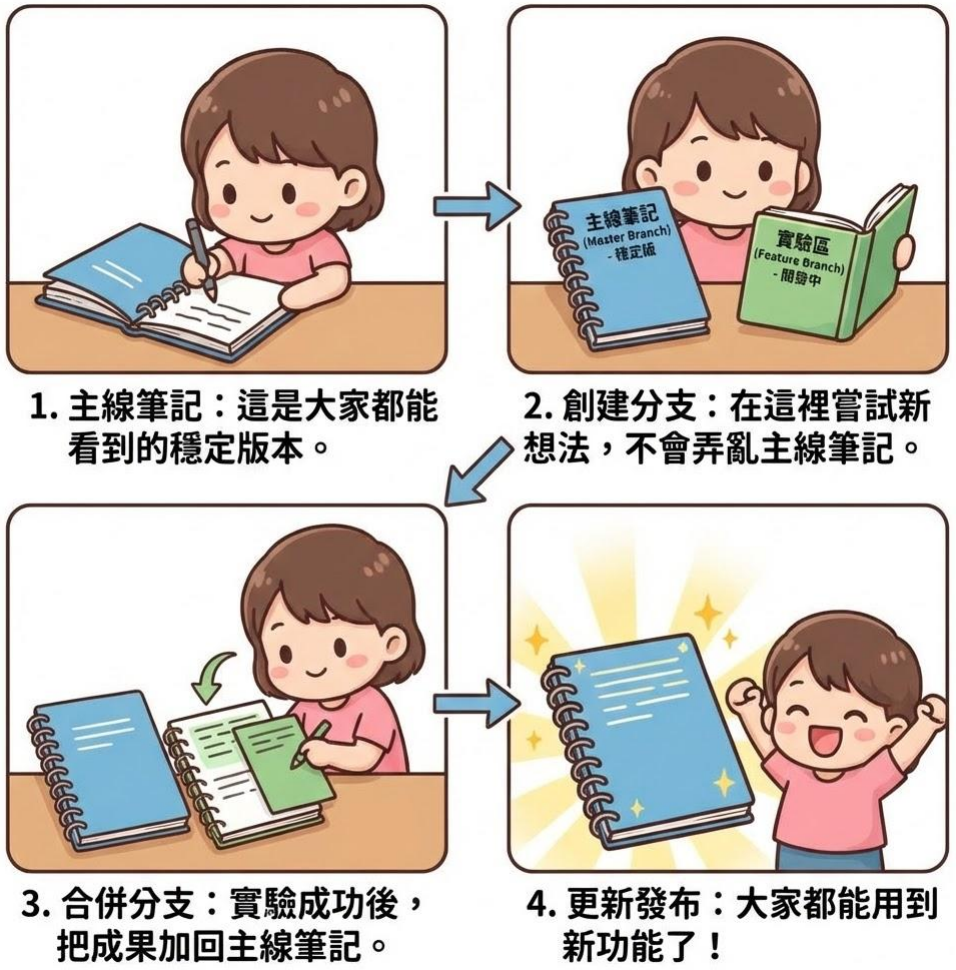
Remote Repositories

Branch (分支)

專案的「平行時空」

可以開一個分支去嘗試新功能，寫壞了也不會影響到原本穩定運作的主線 (Default Branch)。

協作重點： 不同組員可以在不同分支開發，最後再合併在一起。



2. 在 Antigravity 中操作 Git

圖形操作

步驟一：初始化專案（Initialize）

開啟版本控制開關

開啟面板： 點擊左側邊欄的 「存放庫 (Source Control)」 圖示（看起來像樹枝分岔的圖標）。

執行： 點擊 **"Initialize Repository"** 按鈕。

結果： 資料夾中會建立 .git 紀錄點，Git 開始監視您的檔案變更。

步驟二：暫存與提交（Stage & Commit）

1. 暫存 (Stage)

點擊檔案旁邊的 `+` 號。

這代表：「我準備要把這個檔案加入下一個存檔點」。

2. 提交 (Commit)

在輸入框寫下訊息（例如：修復首頁 UI）。

點擊 **Commit** 按鈕（或打勾圖示）。

為什麼 Commit Message 很重要？

好的訊息是給「未來的自己」與「團隊成員」看的。

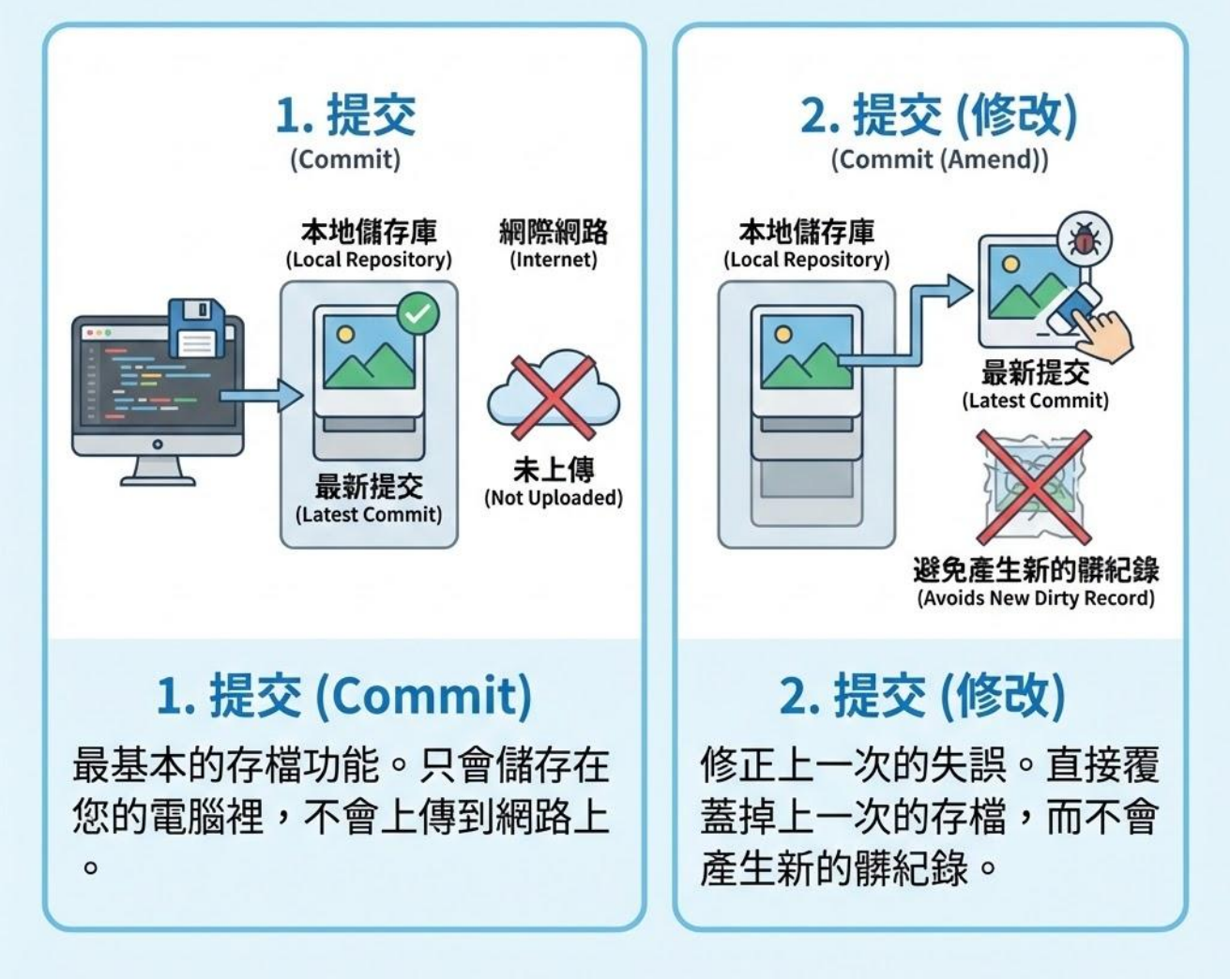
不良示範： "update", "fix", "123", "aaaa"

良好示範： "完成首頁設計"、"修復登入按鈕點擊無反應"

優質訊息公式：

[動作] + [目標名稱] + [結果描述]

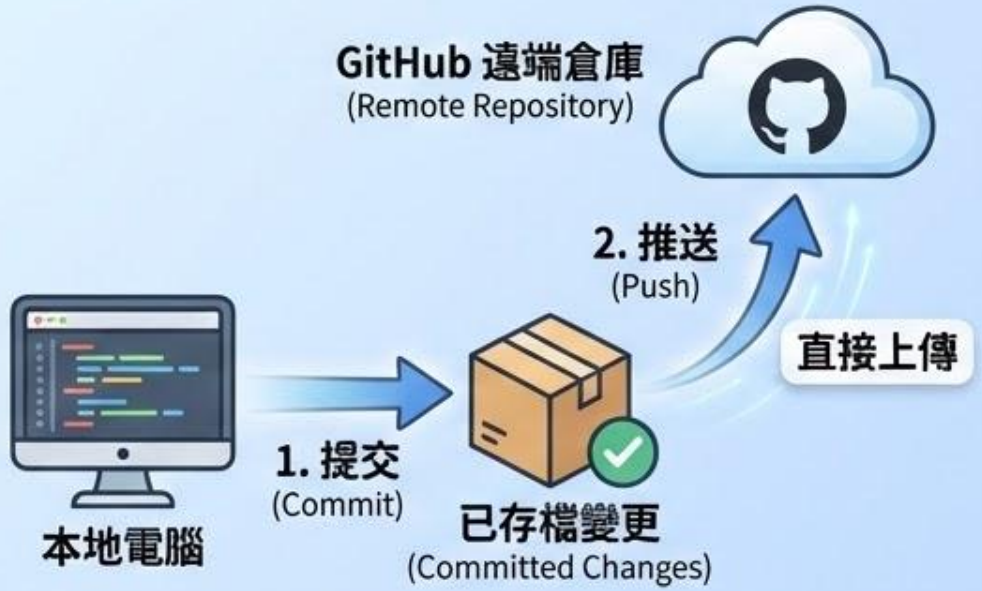
提交按鈕選單：修正與單機



提交按鈕選單：同步專家

圖解 Git 同步流程

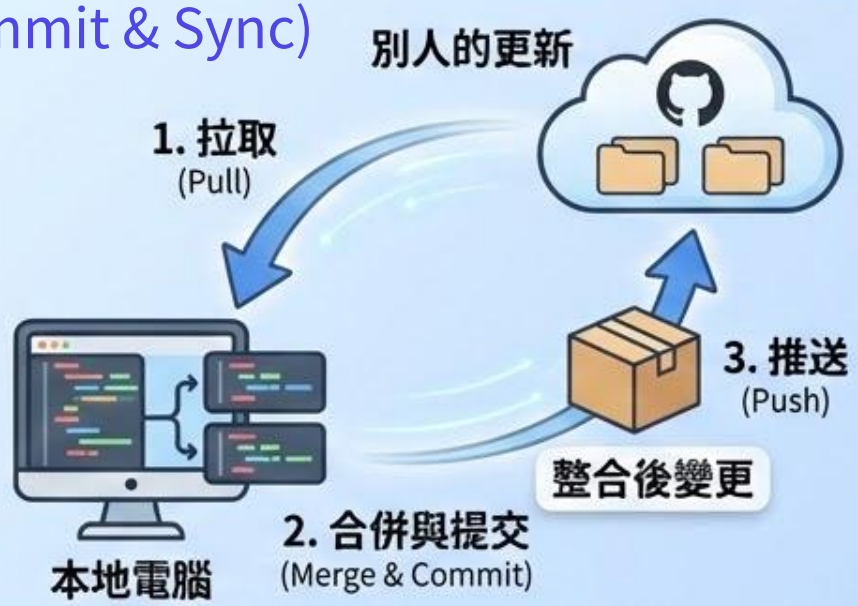
3. 提交與推送 (Commit & Push)



存檔後，自動幫您上傳到 GitHub。省去多按一次 Push 的步驟。

4. 提交與同步 (Commit & Sync)

團隊開發最推薦！



會先下載別人的更新 **(Pull)**，整理好後後再上傳您的。避免衝突。

同步到雲端（Push & Pull）

Publish Branch / Sync Changes：將進度上傳到 GitHub，確保電腦壞掉程式碼也不會丟失。

常見作法

1. 頻繁 Commit

不要寫了一整天才存檔，建議每完成一個小功能 (甚至是畫好一個按鈕) 就 Commit 一次。

2. 訊息要清楚

拒絕 "fix", "update"。明確告知變動內容，方便日後排除 Bug。

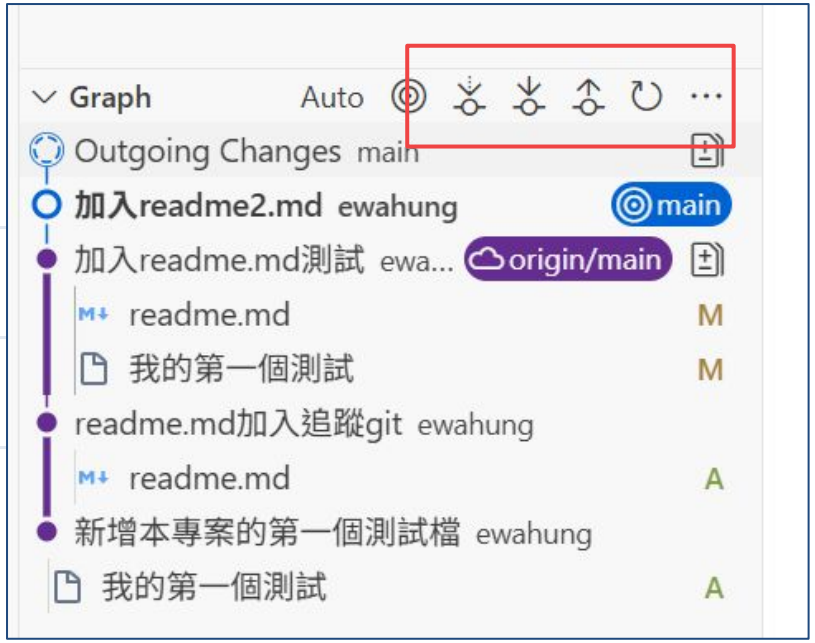
3. 先 Pull 再 Push

要上傳前，先拉取同學的更新，確保您是在最新的基礎上工作，降低衝突風險。

良好溝通： 若要動到大家都會用的核心檔案 (如 App.tsx)，先在群組告知。

工具列小圖示 說明 (上排)

圖示	功能名稱	詳細說明
↓	擷取 (Fetch)	只檢查遠端更新，不會改動您的程式碼。
↓	提取 (Pull)	下載遠端更新並直接合併到您的程式碼。
↑	推送 (Push)	將您的本地存檔上傳至遠端。



同步按鈕位置（狀態列）

- main：您目前正在處理的分支名稱。
- ↻ (旋轉箭頭)：同步圖示。點擊它通常會同時執行「拉取 (Pull)」與「推送 (Push)」。
- 0↓ (向下箭頭)：代表雲端有 0 個進度是您本地還沒有的（Incoming Changes）。
- 1↑ (向上箭頭)：代表本地有 1 個進度是雲端還沒有的（Outgoing Changes）。

3. Git 常見指令

Terminal 終端機進階操作

Terminal 常見指令 (1/2)

動作	指令 (Command)	說明
初始化	git init	在目前資料夾建立新的 Git 倉庫。
下載專案	git clone [URL]	從 GitHub 下載整份專案。
查看狀態	git status	列出被修改但還沒存檔的檔案。
暫存檔案	git add .	將所有修改加入準備存檔區。

Terminal 常見指令 (2/2)

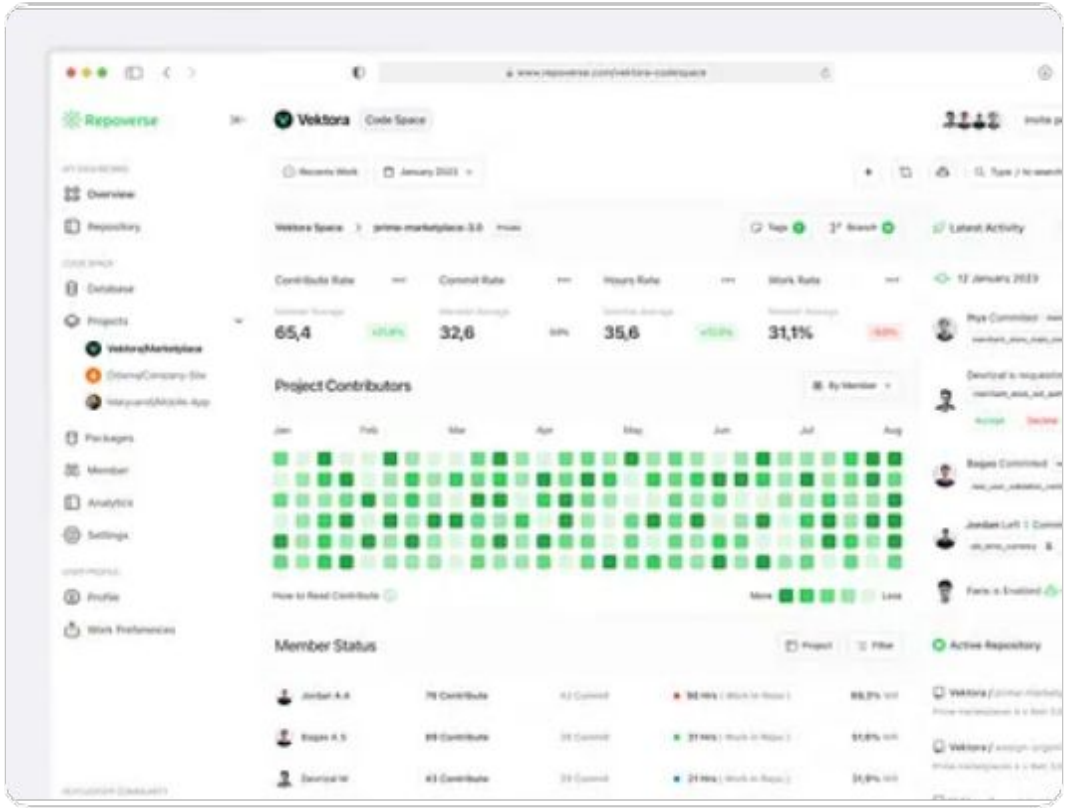
動作	指令 (Command)	說明
提交存檔	git commit -m "訊息"	建立一個具備說明的版本紀錄。
上傳雲端	git push	將進度推送到伺服器。
下載更新	git pull	將他人進度拉回本地並合併。
切換分支	git checkout [Name]	切換至不同開發分支。

4. 如何回到過去 (切換版本)

情境 A：救回某個檔案

Timeline (時間軸) 功能：

- 在左側檔案總管選取檔案。
- 展開下方 **TIMELINE** (時間表)。
- 右鍵點擊紀錄 → **「開啟變更 (Open Changes)」**。
- 將左側舊版本內容複製貼回目前的檔案。



情境 B：救回被右鍵刪除的檔案

方法：捨棄變更 (GUI)

1. 前往「存放庫」面板。
2. 找到標記為 `D` **(Deleted)** 的灰色檔案。
3. 按右鍵選擇 **「捨棄變更 (Discard)」** 或點擊 ↶ 圖示。

方法：Terminal 指令

```
git restore [檔案名稱]
```

檔案會立即重新出現在資料夾中。

情境 C：不小心提交了刪除動作？

若檔案已從變更清單消失，代表刪除已被 Commit。請這樣做：

1. 在 **COMMITS 圖表** 中點擊「刪除之前」的版本。
2. 在顯示的檔案清單中找到該檔案。
3. 按右鍵 → **「開啟 (Open File)」**。
4. 複製內容，手動在資料夾新建同名檔案並貼上。

5. 資料夾與 Git 串接

情境 A：全新的本地專案

流程摘要

1. File → Open Folder 開啟專案。
2. 點擊 Source Control 圖示。
3. 按下 **「Initialize Repository」**。
4. 進行第一次 Commit。

情境 B：本地連接到 GitHub

1. GitHub 端設定

- 點擊右上角 「+」 → New repository。
- 輸入名稱，❌ **不要勾選** README 或 .gitignore。
- 複製 HTTPS 網址。

情境 C：如何下載網路上的專案

GUI 快速鍵：Ctrl + Shift + P

1. 輸入：`Git: Clone`。
2. 貼上專案網址。
3. 選擇電腦存放位置。
4. 下載後點擊「開啟」，並執行 `npm install`。

如何判斷資料夾是否已有 Git？

視覺法

1. 左下角有分支名 (main)。
2. 資料夾內有隱藏的 **.git** 資料夾。

指令法

執行 git status。

若顯示錯誤代表沒有 Git。

6. 分組協作每日工作流程

確保團隊零衝突的高效工作法

📥 開始工作前：務必先 Pull

建立健康的開工習慣

1. 開啟 Antigravity 專案。
2. 立即點擊同步/拉取 (Pull)。
3. 確認您的程式碼已獲得同學最新的改動。

工作期間：頻繁且專一

- **不要一次改太多檔案：** 分散工作量，降低衝突機率。
- **頻繁 Commit：** 即使是很小的功能也要存檔。
- **良好溝通：** 若要動到大家都會用的核心檔案 (如 App.tsx)，先在群組告知。

結束工作後：確保上傳成功

1. Commit 所有最後的變更。
2. 使用**「同步處理 (Sync)」**。
3. 前往 GitHub 網頁檢查提交歷史。確認綠色亮點已出現！

7. gitignore

排除不必要的檔案，保持專案乾淨

什麼是 .gitignore？

Git 的過濾器

這是一個純文字檔，告訴 Git「哪些檔案絕對不要追蹤」。

優點：

- 加速上傳速度。
- 保護敏感資訊 (如密碼)。
- 避免上傳龐大的系統暫存檔。

8. 常見錯誤與解決方法

不再被 Git 報錯嚇到

報錯 A：進度落後或未提交

❌ Behind origin/main

原因： 雲端有新的存檔，您手上的太舊了。

解法： 點擊藍色按鈕「同步變更」。

❌ Please commit or stash

原因： 您手上有些修改還沒存檔，Git 不敢讓您下載別人的更新。

解法： 先 Commit 您目前的進度，然後再 Pull。

報錯 B：拒絕更新或非 Git 目錄

❌ Updates were rejected

原因： 同學已上傳新東西，您沒 Pull 就想 Push。

解法： 先 Pull → 解決衝突(如果有) → 再 Push。

❌ not a git repository

原因： 所在資料夾沒有 Git 指南檔。

解法： 前往面板點擊「Initialize Repository」。

9. 衝突處理 (Merge Conflict)

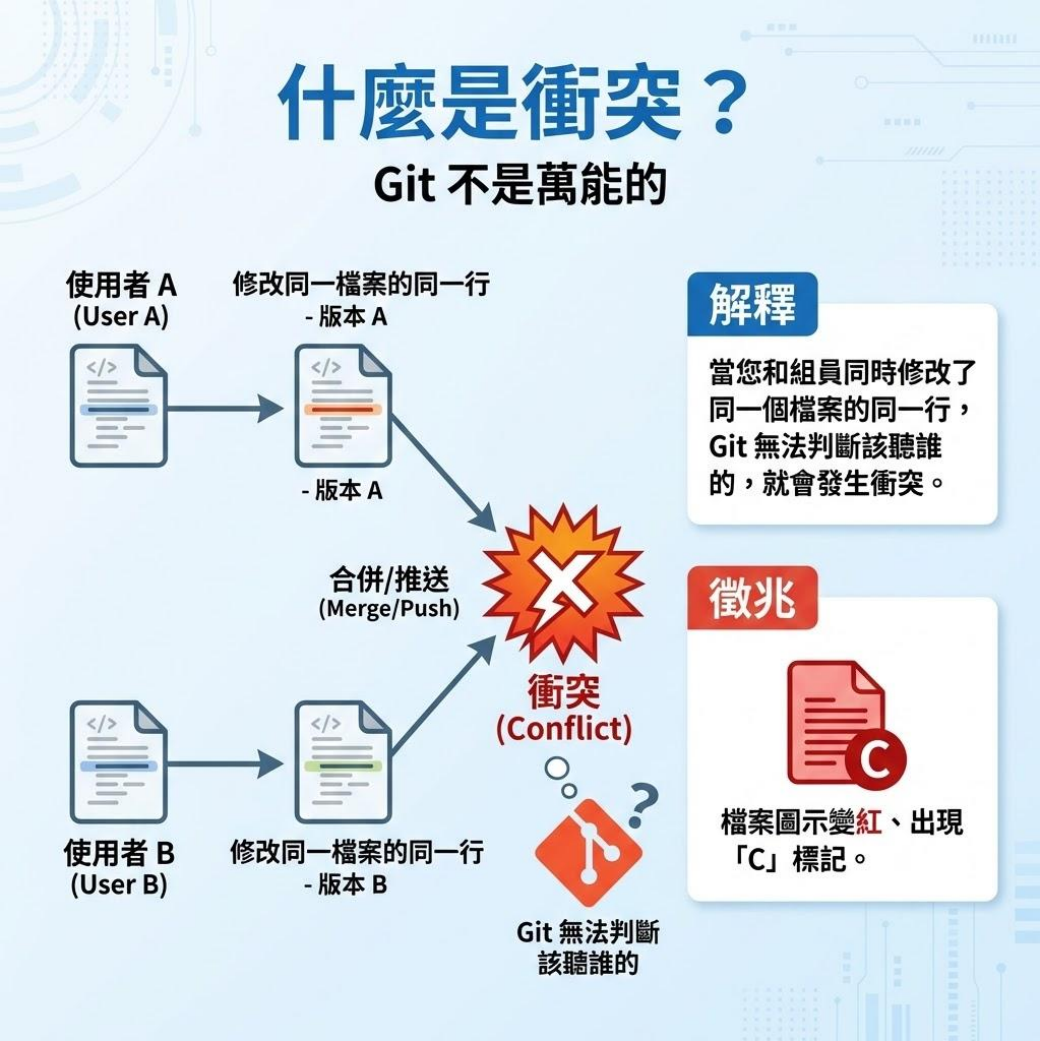
當平行時空交會時的碰撞處理

什麼是衝突？

Git 不是萬能的

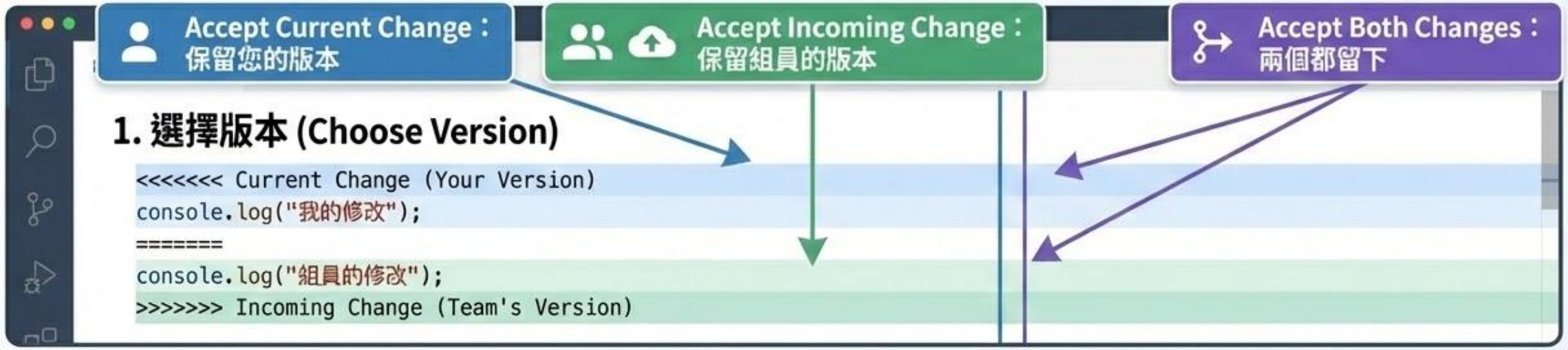
當您和組員**同時修改了同一個檔案的同一行**，Git 無法判斷該聽誰的，就會發生衝突。

徵兆： 檔案圖示變紅、出現 **「C」** 標記。



如何解衝突？

編輯器內的魔法按鈕 & 解決流程



2. 存檔與暫存 (Save & Stage)

存檔 (Save)
選完後，請記得儲存檔案 (Ctrl+S / Cmd+S)。

暫存變更 (Stage Changes)
在原始碼管理面板點擊 `+` 號。

3. 提交完成 (Commit & Finish)

提交 (Commit)
最後輸入訊息並點擊提交按鈕，衝突即告解決！

11. 補充：進階操作

⚠️ 永久回退專案（慎用！）

此操作會**永久刪除**紀錄點之後的所有改動。

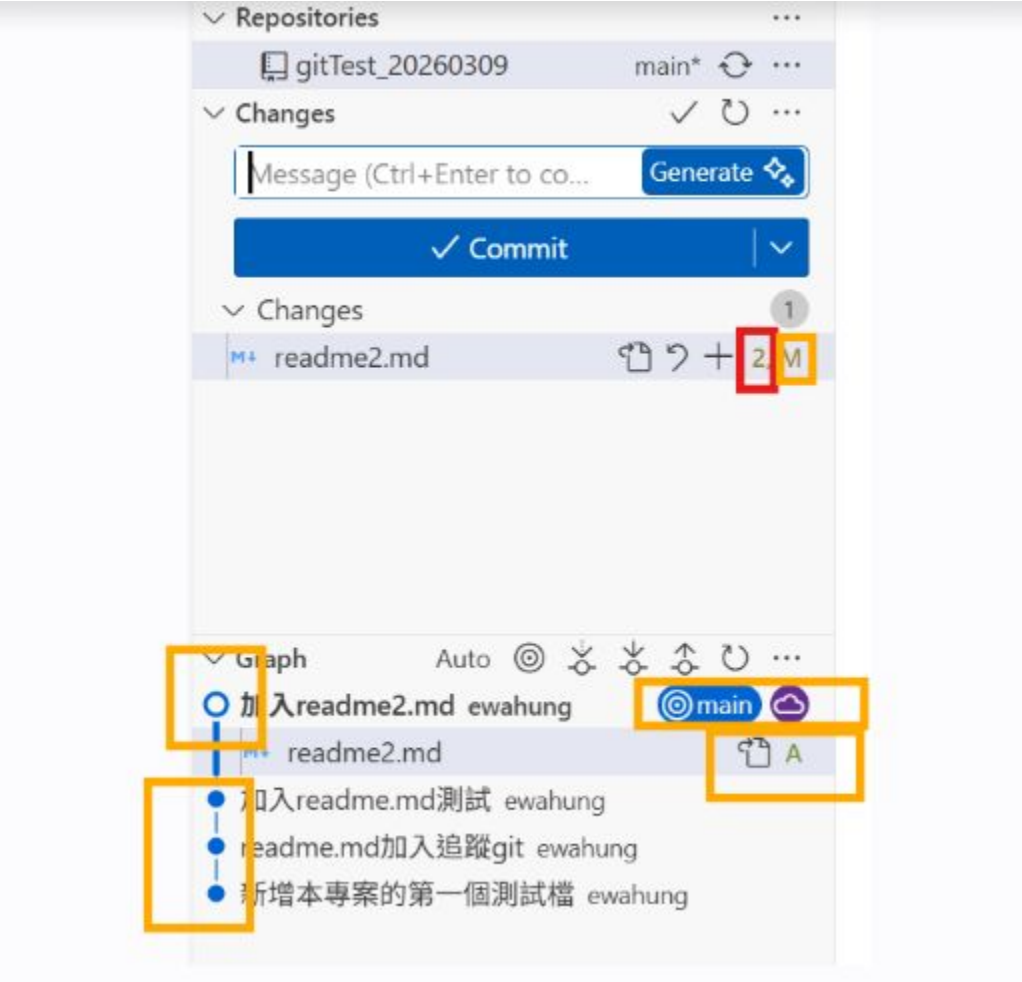
```
git reset --hard [Commit ID]
```

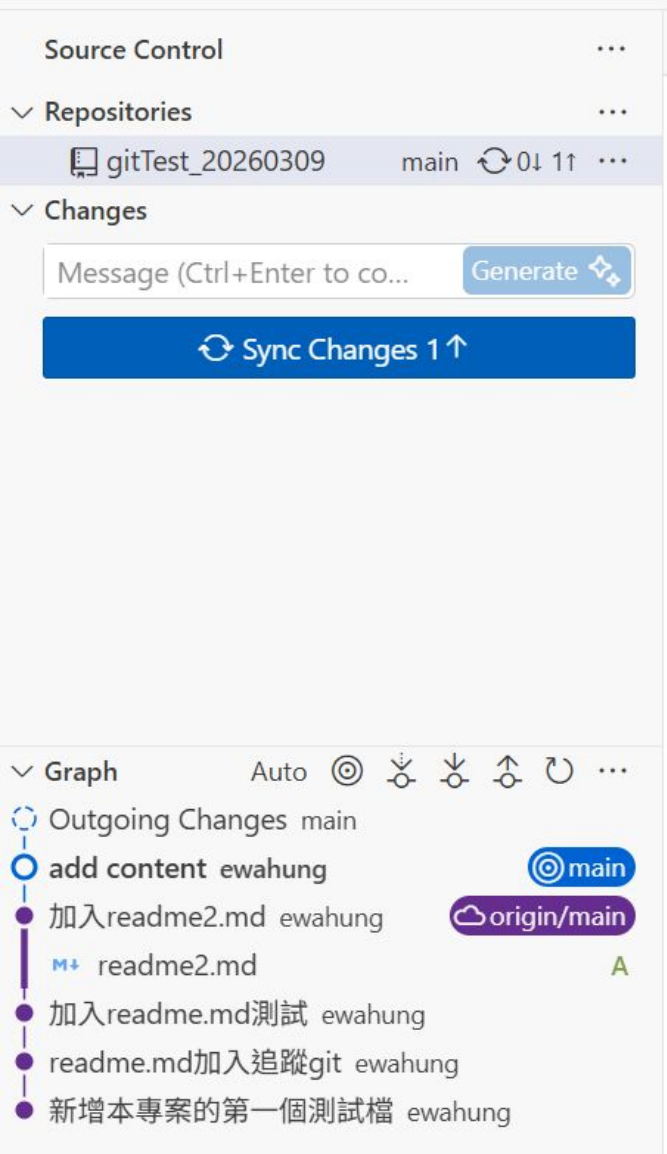
除非您確定後面寫的全都是垃圾，否則請不要隨便使用 reset --hard！

⚠️ 永久回退專案 (慎用！)

除非您確定後面寫的全都是垃圾，否則請不要隨便使用 reset --hard！

⚠️ 想一想


Repositories
gitTest_20260309
main*
Changes
Message (Ctrl+Enter to co...
Generate
Commit
Changes
1
readme2.md
2, M
Graph
Auto
加入readme2.md ewahung
main
readme2.md
A
加入readme.md測試 ewahung
readme.md加入追蹤git ewahung
新增本專案的第一個測試檔 ewahung


Source Control
Repositories
gitTest_20260309
main
0↓ 1↑
Changes
Message (Ctrl+Enter to co...
Generate
Sync Changes 1↑
Graph
Auto
Outgoing Changes main
add content ewahung
main
加入readme2.md ewahung
origin/main
readme2.md
A
加入readme.md測試 ewahung
readme.md加入追蹤git ewahung
新增本專案的第一個測試檔 ewahung

作業一：上傳 Week 2 成果至git及學校系統

作業一

1. 在 Antigravity 整理你的第一個作業： 學號+姓名.md。
2. 註冊 GitHub
3. 建立 Lesson01 倉庫。
4. 初始 Git Repositories > Commit 作業一 > 推送 (Push) 上傳。（Local vs Remote Repositories需同步）
5. ch01 上教材，進行Commit，並進行同步 （Local vs Remote Repositories需同步）
6. ch02 上教材，進行Commit，並進行同步 （Local vs Remote Repositories需同步）
7. ch03 上教材，進行Commit，並進行同步 （Local vs Remote Repositories需同步）
6. 繳交方式： md檔 + antigravity 內 github graph 畫面上傳至教學平台